Konfiguracja środowiska

In [1]:
```
!pip install transformers datasets evaluate accelerate sacremoses -
!pip install bitsandbytes peft -q  # dla adapterów i LoRA
```

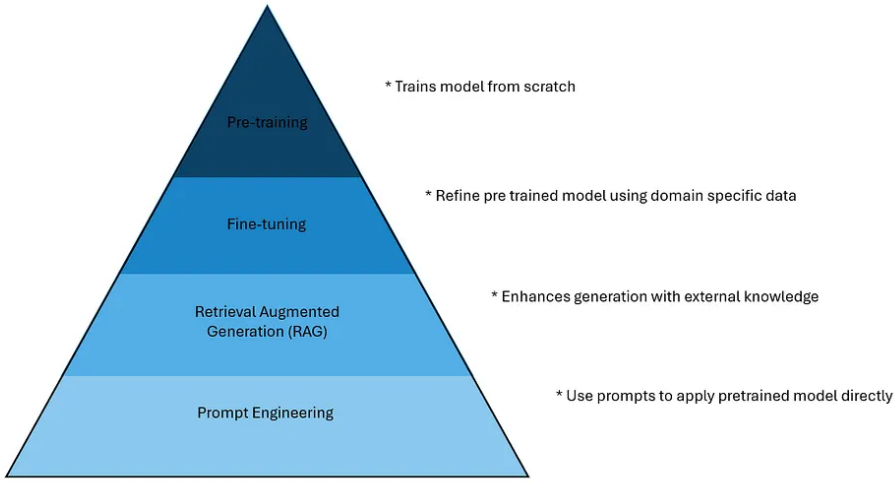
```
[notice] A new release of pip is available: 24.3.1 -> 26.3
[notice] To update, run: pip install --upgrade pip

[notice] A new release of pip is available: 24.3.1 -> 26.3
[notice] To update, run: pip install --upgrade pip
```

In [2]:
```
from transformers import AutoTokenizer, AutoModelForSequenceClassif
from datasets import Dataset
from datasets import load_dataset
import numpy as np
import evaluate
import torch
```

```
~/PJN_s31180/.venv/lib/python3.12/site-packages/tqdm/auto.py:21: Tqd
mWarning: IProgress not found. Please update jupyter and ipywidgets.
See https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
W1111 19:32:50.882000 42403 torch/distributed/elastic/multiprocessin
g/redirects.py:35] NOTE: Redirects are currently not supported in Ma
cOs.
```

Czym różni się pretraining, fine-tuning, RAG i prompt-tuning?



Pretraining

czyli trenowanie modelu językowego od zera

In [3]:
```
from transformers import BertConfig, BertForMaskedLM
```

```
# Tworzymy pusty model
config = BertConfig(
    vocab_size=30522,
    hidden_size=256,
    num_hidden_layers=4,
    num_attention_heads=4,
    intermediate_size=512
)

model = BertForMaskedLM(config)
```

Model uczy się ogólnych zależności językowych, np. jakie słowa występują razem. Nie ma tu żadnych etykiet ani konkretnego zadania - ten etap jest często bardzo kosztowyny. Z powodu zasobów na zajęciach nie pretrenujemy modeli

Fine-tuning

czyli dostosowanie do konkretnego zadania (np. klasyfikacji sentymentu)

In [4]:
```
import socket

_orig_getaddrinfo = socket.getaddrinfo

def _ipv4_only(host, port, family=0, *args, **kwargs):
    return _orig_getaddrinfo(host, port, socket.AF_INET, *args, **k

socket.getaddrinfo = _ipv4_only
```

In [5]:
```
from transformers import AutoTokenizer, AutoModelForSequenceClassif
from datasets import load_dataset
import numpy as np
import evaluate

# Wczytanie danych (IMDb)
dataset = load_dataset("stanfordnlp/imdb")
small_train = dataset["train"].shuffle(seed=42).select(range(2000))
small_test = dataset["test"].shuffle(seed=42).select(range(500))

# Tokenizacja
tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased"

def tokenize(batch):
    return tokenizer(batch["text"], truncation=True, padding="max_l

train_enc = small_train.map(tokenize, batched=True)
test_enc = small_test.map(tokenize, batched=True)

model = AutoModelForSequenceClassification.from_pretrained("distilb
accuracy = evaluate.load("accuracy")

def compute_metrics(eval_pred):
    logits, labels = eval_pred
```

```
    preds = np.argmax(logits, axis=-1)
    return accuracy.compute(predictions=preds, references=labels)

# Trening
training_args = TrainingArguments(
    output_dir="./fine_tuning",
    num_train_epochs=1,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    eval_strategy="epoch",
    report_to="none"
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_enc,
    eval_dataset=test_enc,
    processing_class=tokenizer,
    compute_metrics=compute_metrics,
)

trainer.train()
trainer.evaluate()
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Ple
ase set a HF_TOKEN to enable higher rate limits and faster download
s.
Loading weights: 100%|██████████| 100/100 [00:00<00:00, 6467.80it/s]
[transformers] DistilBertForSequenceClassification LOAD REPORT from:
distilbert-base-uncased
Key                     | Status     |
------------------------+------------+-
vocab_transform.weight  | UNEXPECTED |
vocab_layer_norm.weight | UNEXPECTED |
vocab_layer_norm.bias   | UNEXPECTED |
vocab_projector.bias    | UNEXPECTED |
vocab_transform.bias    | UNEXPECTED |
classifier.bias         | MISSING    |
classifier.weight       | MISSING    |
pre_classifier.bias     | MISSING    |
pre_classifier.weight   | MISSING    |

Notes:
- UNEXPECTED:   can be ignored when loading from different task/arch
itecture; not ok if you expect identical arch.
- MISSING:      those params were newly initialized because missing
from the checkpoint. Consider training on your downstream task.
~/PJN_s31180/.venv/lib/python3.12/site-packages/torch/utils/data/dat
aloader.py:759: UserWarning: 'pin_memory' argument is set as true bu
t not supported on MPS now, device pinned memory won't be used.
  super().__init__(loader)
```

[250/250 02:15, Epoch 1/1]

Epoch	Training Loss	Validation Loss	Accuracy
1	No log	0.412804	0.826000

```
Writing model shards: 100%|██████████| 1/1 [00:00<00:00,  1.63it/s]
~/PJN_s31180/.venv/lib/python3.12/site-packages/torch/utils/data/dat
aloader.py:759: UserWarning: 'pin_memory' argument is set as true bu
t not supported on MPS now, device pinned memory won't be used.
  super().__init__(loader)
~/PJN_s31180/.venv/lib/python3.12/site-packages/torch/utils/data/dat
aloader.py:759: UserWarning: 'pin_memory' argument is set as true bu
t not supported on MPS now, device pinned memory won't be used.
  super().__init__(loader)
```

[63/63 00:06]

Training Loss	Validation Loss	Epoch	Accuracy
No log	0.412804	1	0.826000

Out[5]: `{'eval_loss': 0.4128040671348572, 'eval_accuracy': 0.826}`

Uczymy ten sam model na nowym zadaniu (tu: klasyfikacja sentymentu). Całe wagi modelu mogą być aktualizowane, lub tylko część.

In [6]:
```
texts = [
    "I really enjoyed this movie, it was awesome!",
    "This film was boring and too long."
]

inputs = tokenizer(texts, padding=True, truncation=True, return_ten

outputs = trainer.model(**inputs)

import torch
preds = torch.nn.functional.softmax(outputs.logits, dim=-1)
labels = torch.argmax(preds, dim=1)

print(preds)
print(labels)
```

```
tensor([[0.0513, 0.9487],
        [0.9496, 0.0504]], device='mps:0', grad_fn=<SoftmaxBackward0
>)
tensor([1, 0], device='mps:0')
```

RAG

RAG = Retrieval-Augmented Generation Model generatywny (np. BERT, GPT, T5) ma dostęp do zewnętrznej bazy wiedzy (dokumentów, notatek, artykułów):

1. Retriever wyszukuje najbardziej podobne fragmenty tekstu,

2. Generator wykorzystuje je do stworzenia odpowiedzi.

O RAGach już niedługo :)

Prompt-tuning

czyli użycie gotowych modeli bez ich trenowania, tylko z pomocą promptów

In [7]:
```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

t5_tokenizer = AutoTokenizer.from_pretrained("google/flan-t5-small"
t5_model = AutoModelForSeq2SeqLM.from_pretrained("google/flan-t5-sm

def generator(prompt, max_new_tokens=20):
    inputs = t5_tokenizer(prompt, return_tensors="pt")
    output = t5_model.generate(**inputs, max_new_tokens=max_new_tok
    return [{"generated_text": t5_tokenizer.decode(output[0], skip_

prompt_good = "Classify the sentiment of the following movie review
result = generator(prompt_good, max_new_tokens=20)

print(result)
```

```
Loading weights: 100%|██████████| 190/190 [00:00<00:00, 4465.00it/s]
[transformers] The tied weights mapping and config for this model sp
ecifies to tie shared.weight to lm_head.weight, but both are present
in the checkpoints with different values, so we will NOT tie them. Y
ou should update the config with `tie_word_embeddings=False` to sile
nce this warning.
[{'generated_text': 'positive'}]
```

In [8]:
```
prompt_bad = "This movie was great and full of emotions. What do yo
print(generator(prompt_bad, max_new_tokens=30))
```

```
[{'generated_text': "I'm not sure if this movie is a good one or a b
ad one. It's a good movie. It'"}]
```

Podsumowanie

Aspect	Prompt Engineering	Retrieval Augmented Generation (RAG)	Fine-tuning	Pre-training LLMs
Purpose	Guides model with specific prompts	Enhances generation with external knowledge	Tailors model to specific task	Trains model from scratch
Approach	Provides structured input prompts	Combines retrieval and generation stages	Adjusts existing model weights	Trains model on diverse data
Training Data	Prompt-template pairs	Large text corpora + structured data	Specific task data	General text corpora
Knowledge Incorporation	Incorporated through prompts	Uses retrieved knowledge during generation	Adjusts model for task-specific	Learns knowledge during training
Adaptability to New Tasks	Moderate	Highly adaptable due to retrieval	Limited to related tasks	Requires re-training or fine-tuning
Data Efficiency	Efficient use of prompt-data pairs	Utilizes external data efficiently	Focuses on task-specific data	Requires large and diverse data
Training Time	Shorter due to focused prompts	Longer due to dual-stage training	Shorter compared to pre-training	Longest due to training from scratch
Model Size	May be larger due to prompts	May have larger footprint due to retrieval	Smaller after fine-tuning	Initially larger but can be compressed
Performance on Unseen Data	Generally robust if prompts cover scenarios well	Generally robust due to external knowledge	May generalize well if tasks are similar	Depends on pre-training quality
Resource Requirements	Moderate due to focused data	Higher due to dual-stage training	Moderate due to fine-tuning	Highest due to extensive training

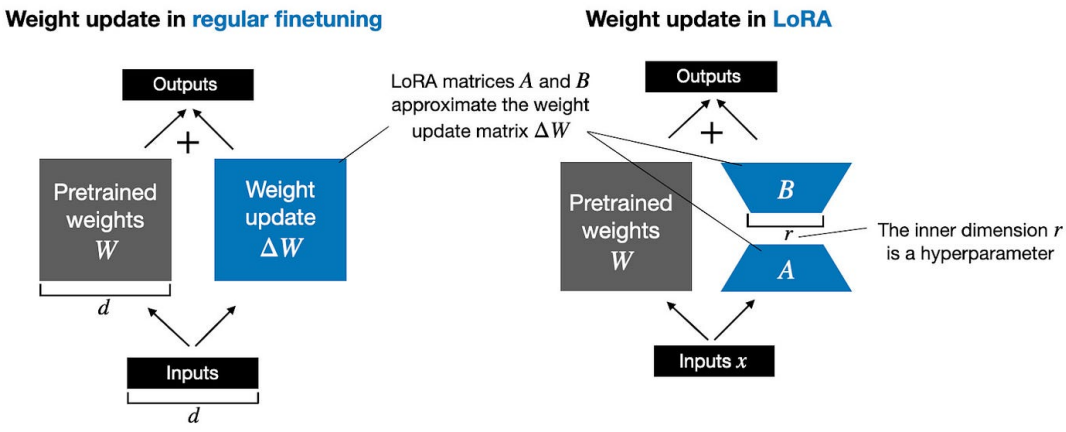
> Więcej o tych 4 metodach:
> https://medium.com/@jainpalak9509/use-llms-pre-training-fine-tuning-rag-and-prompt-engineering-564d5670f44d

LoRA

LoRA (Low-Rank Adaptation) to metoda efektywnego dostrajania (fine-tuningu) dużych modeli językowych. Zamiast aktualizować wszystkie miliony parametrów modelu (np. GPT, BERT), LoRA wprowadza małe, dodatkowe macierze, które uczą się zmian, a oryginalne wagi modelu pozostają zamrożone.

Jak działa LoRA?

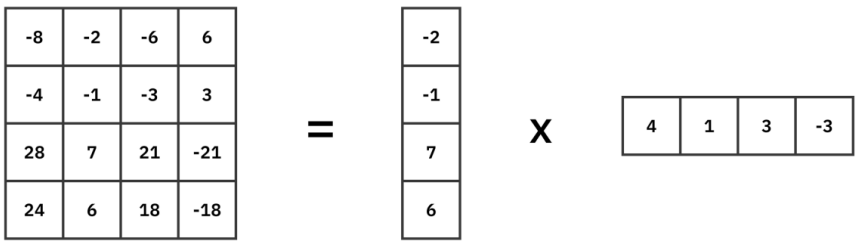
Wyobraź sobie, że masz już gotowy model (np. BERT), który zna język ogólnie, ale chcesz, żeby lepiej rozumiał np. język prawniczy.



LoRA zamraża oryginalne parametry modelu, a następnie dodaje małe,

dodatkowe macierze (A i B), które uczą się nowych informacji.

Macierz Weight update może być rozbita w ten sposób:



Przykład: Mamy model, gdzie delta W ma 10 000 rzędó i 20 000 kolumn. To daje nam razem 200 000 000 parametrów - dużo!

Możemy zaplanować, że nasza macierz A będzie miała 10 000 rzędó na r = 8 kolumn. Wtedy B ma r = 8 rzedó i 20 000 kolumn. W tej sytuacji mamy 10 000 * 8 + 20 000 * 8 parametrów = 240 000 parametrów (około 830 raza mniej niż na początku!)

Na końcu, po treningu, LoRA łączy te małe poprawki z oryginalnym modelem — otrzymujemy nową, lepiej dopasowaną wersję modelu.

Zalety	Ograniczenia
Szybszy trening	Nadal wymaga pewnych zasobów GPU
Mało pamięci (uczy tylko małe macierze)	Nie zawsze działa równie dobrze jak pełny fine-tuning
Możliwość łączenia kilku LoRA z jednym modelem bazowym	Trudniejsze debugowanie (więcej warstw pośrednich)

> Więcej o LoRA: https://www.ibm.com/think/topics/lora

Zadanie

🎯 Użyj LoRA do zfine tunowania modelu na datasetcie

"Abirate/english_quotes".

1. Model: `model_name = "distilgpt2"`
2. Zanim dodasz LoRa musimy zamrozić parametry modelu (Podpowiedź: coś musi mieć requires_grad na False lub możemy użyć mark_only_lora_as_trainable)
3. Konfiguracja LoRA:

- używamy LoraConfig z biblioteki peft
- Musimy ustawić r = 8 (wymiar low-rank)

4. Używamy datasetu o angielskich cytatach: data = `load_dataset("Abirate/english_quotes")` (Podpowiedź: będzie trzeba ztokenizować ten zbiór ;))
5. Trenujemy przy użyciu TrainingArguments i Trainer. Ustawimy nie epoki a kroki: `max_steps=1000`
6. Przetestuj wytrenowany model

In [9]:
```
from transformers import AutoModelForCausalLM
from peft import LoraConfig, get_peft_model

#Model
model_name = "distilgpt2"

tokenizer = AutoTokenizer.from_pretrained(model_name)
tokenizer.pad_token = tokenizer.eos_token
model = AutoModelForCausalLM.from_pretrained(model_name)

for param in model.parameters():
    param.requires_grad = False

config = LoraConfig(
    r=8,
    lora_alpha=16,
    lora_dropout=0.05,
    task_type="CAUSAL_LM"
)

# TODO - zaaplikuj LoRA na modelu
model = get_peft_model(model, config)

# Sprawdźmy ile parametrów będziemy trenować - tutaj musimy już mie
def print_trainable_parameters(model):
    trainable_params = 0
    all_param = 0
    for _, param in model.named_parameters():
        all_param += param.numel()
        if param.requires_grad:
            trainable_params += param.numel()
    print(f"Parametry do treningu: {trainable_params:,} / {all_para

print_trainable_parameters(model)
```

```
# Data
data = load_dataset("Abirate/english_quotes")

def preprocess_example(example):
    # Musimy połaczyć elementy z datasetu w jedność - qoutes + auth
    example["text"] = example["quote"] + " - " + example["author"]
    return example

def tokenize_fn(examples):
    return tokenizer(examples["text"], truncation=True, padding="ma

data = data.map(preprocess_example)
train_enc = data["train"].map(tokenize_fn, batched=True, remove_col

data_collator = DataCollatorForLanguageModeling(tokenizer=tokenizer

training_args = TrainingArguments(
    output_dir="./lora",
    max_steps=1000,
    per_device_train_batch_size=8,
    learning_rate=2e-4,
    logging_steps=100,
    report_to="none"
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_enc,
    data_collator=data_collator,
)

model.config.use_cache = False
trainer.train()

# Test
prompt = "Training models with LoRa is "
inputs = tokenizer(prompt, return_tensors="pt").to(model.device)

model.eval()
with torch.no_grad():
    output = model.generate(**inputs, max_new_tokens=40, temperatur

print("\n💬 Model output:\n", tokenizer.decode(output[0], skip_spec
```

```
Loading weights: 100%|██████████| 76/76 [00:00<00:00, 7152.86it/s]
~/PJN_s31180/.venv/lib/python3.12/site-packages/peft/tuners/lora/lay
er.py:2760: UserWarning: fan_in_fan_out is set to False but the targ
et module is `Conv1D`. Setting fan_in_fan_out to True.
  warnings.warn(
Parametry do treningu: 147,456 / 82,060,032 (0.18%)
```

```
~/PJN_s31180/.venv/lib/python3.12/site-packages/torch/utils/data/dat
aloader.py:759: UserWarning: 'pin_memory' argument is set as true bu
t not supported on MPS now, device pinned memory won't be used.
  super().__init__(loader)
[transformers] `loss_type=None` was set in the config but it is unre
cognized. Using the default loss: `ForCausalLMLoss`.
```

[1000/1000 07:19, Epoch 3/4]

Step	Training Loss
100	3.985412
200	3.448621
300	3.411850
400	3.351236
500	3.315918
600	3.330230
700	3.288001
800	3.303707
900	3.294485
1000	3.313872

```
~/PJN_s31180/.venv/lib/python3.12/site-packages/torch/utils/data/dat
aloader.py:759: UserWarning: 'pin_memory' argument is set as true bu
t not supported on MPS now, device pinned memory won't be used.
  super().__init__(loader)
[transformers] The following generation flags are not valid and may
be ignored: ['temperature']. Set `TRANSFORMERS_VERBOSITY=info` for m
ore details.
💬 Model output:
 Training models with LoRa is iced up with a lot of the same things.
I like to think that the best way to do this is to use the LoRa mode
l. I like to think that the best way to do this
```